Tutorial 10: Image Segmentation using U-Net (Custom Model)

Laiba Niazi

536672

1. Objectives

The primary goal of this project was to transition a Keras-based U-Net architecture into PyTorch and apply it to a custom-developed dataset. The specific tasks included implementing the architecture from scratch, modifying layer properties for optimization, and evaluating the impact of different learning rates.

2. Dataset and Preprocessing

2.1 Custom Dataset

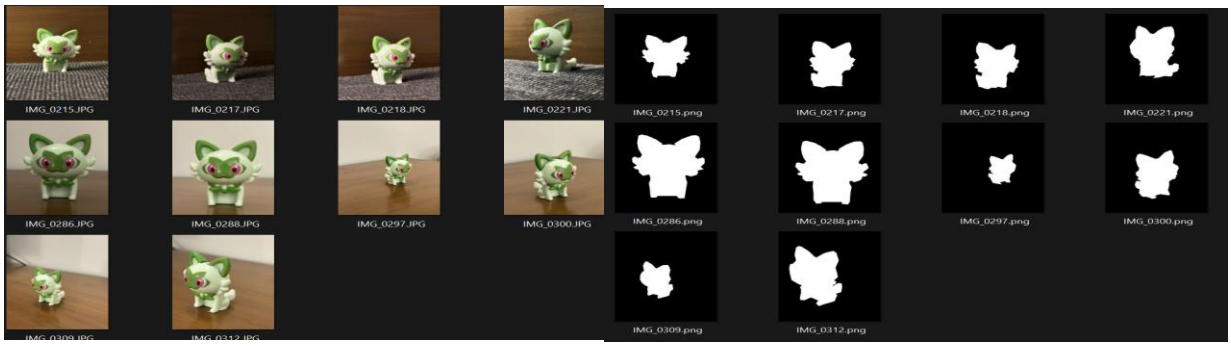
The project utilizes a custom dataset specifically curated for this segmentation task. Unlike standard benchmark datasets, this data was captured and annotated manually to address a specific object detection need.

2.2 Mask Generation

The ground truth masks for this dataset were generated using makesense.ai, an online tool for labeling photos. The annotations were exported in COCO JSON format and subsequently converted into binary PNG masks for model training.

The preprocessing pipeline involved:

- Converting polygon segmentations from JSON into filled binary silhouettes.
- Resizing both images and masks to a uniform dimension of 128x128 pixels.
- Normalizing pixel values to a range of [0, 1].



3. Methodology

3.1 Model Architecture

The model implements a modified U-Net architecture, characterized by its symmetrical "U" shape consisting of a contracting path (encoder) and an expansive path (decoder).

Key modifications in this implementation include:

- **Batch Normalization:** Added after each convolutional layer to improve training stability and speed.
- **Encoder:** Two main convolutional blocks with 64 and 128 filters, followed by Max Pooling layers.
- **Bottleneck**: A central block with 256 filters to capture high-level features.
- **Decoder**: Transposed convolutions used for upsampling, concatenated with skip connections from the encoder to retain spatial information.
- **Activation**: ReLU is used in hidden layers, and a Sigmoid activation is applied to the final 1x1 convolution to produce a probability map.

4. Training and Evaluation

4.1 Training Configuration

The model was trained using the Adam optimizer and Binary Cross-Entropy (BCE) Loss, which is optimal for binary segmentation tasks.

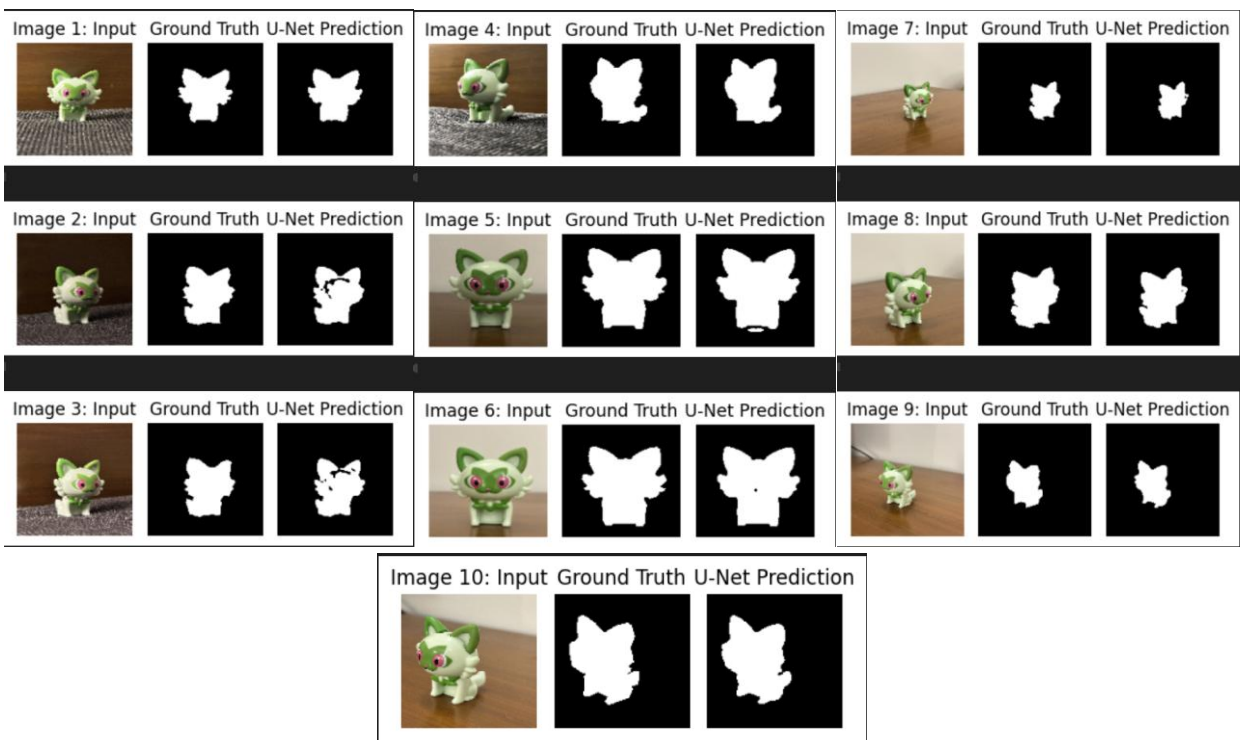
4.2 Hyperparameter Tuning: Learning Rates

Three distinct learning rates were tested to determine the optimal speed of convergence:

1. **0.01:** Rapid initial descent but prone to instability.
2. **0.001:** Balanced convergence.
3. **0.0001:** Slow and steady descent, requiring more epochs for full convergence.

5. Final Segmentation Results

The final model effectively identified the boundaries of the figurine. Despite the limited sample size (10 images), the addition of Batch Normalization allowed the model to produce clear binary masks.



6. Conclusion

The custom U-Net implementation successfully performed segmentation on the provided custom dataset. Through experimentation, it was found that the choice of learning rate

significantly impacts the model's ability to reach a global minimum. The use of Batch Normalization further assisted in stabilizing the training process, leading to clean predicted masks that closely mirror the ground truth labeled via makesense.ai.